

2LIG IHEC Carthage

Spring Boot StyleHub Tutorial

Full Training Guide

HTML • CSS • Spring Boot MVC • Thymeleaf • JPA • MySQL

PART	TOPIC
Part 1 (2.5h)	Spring Boot Application — No Database (MVC Architecture, Thymeleaf, Static Data)
Part 2 (2.5h)	Database Integration — Spring Data JPA, MySQL, CRUD Operations, Relationships

Prerequisites: Java 17+, Maven, IDE (IntelliJ/VS Code), Basic HTML/CSS Knowledge

PART 1

Spring Boot MVC — No Database

Estimated Time: 2.5 Hours

1.1 What is Spring Boot?

Spring Boot is a Java framework that makes it easy to build production-ready web applications. It eliminates the need for extensive configuration by providing sensible defaults and auto-configuration. Think of it as a 'batteries included' version of the Spring Framework.

Concept	Explanation
Spring Framework	Core Java framework for dependency injection and application structure
Spring Boot	Opinionated layer on top of Spring — auto-configures everything
Maven/Gradle	Build tools that manage dependencies (like npm for Node.js)
Thymeleaf	HTML template engine — lets you embed Java data into HTML pages
MVC Pattern	Design pattern: Model (data), View (HTML), Controller (logic)

1.2 The MVC Architecture

MVC stands for Model–View–Controller. It is the backbone of how Spring Boot organizes a web application. Every request follows this exact flow:

 Request Flow in Spring Boot MVC
1. Browser sends HTTP request → http://localhost:8080/products
2. DispatcherServlet receives it (Front Controller — built into Spring Boot)
3. Router maps the URL to a @Controller method
4. Controller calls Service layer to get business logic
5. Service calls Repository to get data (Part 2) or uses static data (Part 1)
6. Controller puts data into the Model (a Map-like object)
7. Controller returns the name of a Thymeleaf View template

8. Thymeleaf renders HTML using Model data
9. Browser receives and displays the final HTML page

The Three Layers Explained

Layer	Role in StyleHub (Clothing E-Commerce)
Model	Represents data: Product (id, name, price, category, imageUrl), Order, Customer
View	HTML pages styled with CSS: home.html, products.html, product-detail.html, cart.html
Controller	Java classes with @Controller annotation that handle URLs like /products, /cart
Service	Business logic layer (sits between Controller and data source)
Repository	Data access layer — talks to database (we use static lists in Part 1)

1.3 Project Setup — Step by Step

We will create our StyleHub project using Spring Initializr, the official project generator for Spring Boot.

1 Go to <https://start.spring.io> in your browser

Field	Value to Enter
Project	Maven
Language	Java
Spring Boot	3.2.x (latest stable)
Group	com.stylehub
Artifact	stylehub
Name	StyleHub
Packaging	Jar
Java Version	17

2 Add Dependencies — Click 'ADD DEPENDENCIES' and search for:

- Spring Web — Enables HTTP web server and REST/MVC
- Thymeleaf — HTML templating engine
- Spring Boot DevTools — Auto-restarts on code changes (dev only)
- Lombok — Reduces Java boilerplate (getters, setters, constructors)

3 Click GENERATE — Download and unzip the project

4 Open in IntelliJ IDEA: File → Open → Select the unzipped folder

5 Wait for Maven to download dependencies, then run the app

Terminal

```
# Run from terminal inside the project folder:
./mvnw spring-boot:run

# You should see:
# Tomcat started on port(s): 8080
# Started StyleHubApplication in 2.3 seconds
```

Success Check

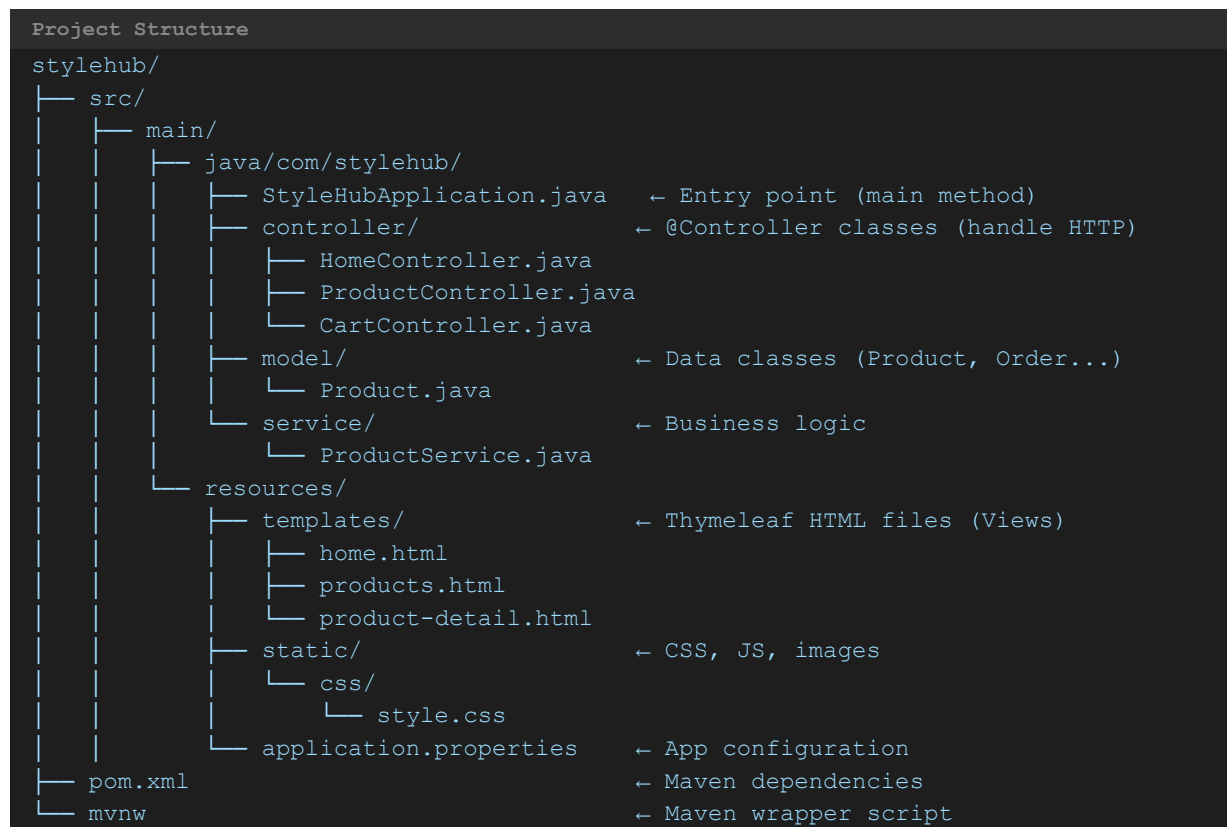
Open browser → <http://localhost:8080>

You should see a Whitelabel Error Page — this is NORMAL at this stage.

It means Spring Boot is running but no routes are defined yet.

1.4 Project Structure Explained

After opening the project, you will see the following folder structure. Every folder has a specific purpose:



1.5 Step 1 — Creating the Model (Product.java)

The Model represents the data of our application. For a clothing e-commerce site, our main entity is a Product. We create a simple Java class to describe what a product looks like.

File Location

Create file at: `src/main/java/com/stylehub/model/Product.java`

```
Java - Product.java
package com.stylehub.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

// @Data generates: getters, setters, toString, equals, hashCode
// @AllArgsConstructor generates a constructor with all fields
// @NoArgsConstructor generates a no-argument constructor
@Data
@AllArgsConstructor
@NoArgsConstructor
public class Product {

    private Long id;           // Unique identifier
    private String name;       // e.g. 'Classic Denim Jacket'
    private String description;
    private double price;      // e.g. 59.99
    private String category;    // 'Men', 'Women', 'Kids'
    private String imageUrl;    // Path to product image
    private boolean inStock;    // Is it available?
    private String size;        // 'S', 'M', 'L', 'XL'

}
```

What is Lombok?

Without Lombok, you would need to manually write 50+ lines of getters, setters, and constructors for every class. Lombok generates all of that automatically at compile time using annotations. Make sure the Lombok plugin is installed in IntelliJ:

Settings → Plugins → Search 'Lombok' → Install → Restart IDE.

1.6 Step 2 — Creating the Service Layer (ProductService.java)

The Service layer contains business logic. It is the 'brain' between the Controller and the data. In Part 1, we use static in-memory data (a Java List) instead of a database.

File Location

Create file at: src/main/java/com/stylehub/service/ProductService.java

```
Java - ProductService.java
package com.stylehub.service;

import com.stylehub.model.Product;
import org.springframework.stereotype.Service;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;

// @Service tells Spring: 'this is a bean - manage it for me'
@Service
public class ProductService {

    // Static product catalog - simulating a database
    private final List<Product> products = Arrays.asList(
        new Product(1L, 'Classic Denim Jacket', 'Timeless blue denim', 59.99,
            'Men', '/images/jacket.jpg', true, 'M'),
        new Product(2L, 'Floral Summer Dress', 'Light chiffon fabric', 44.99,
            'Women', '/images/dress.jpg', true, 'S'),
        new Product(3L, 'Kids Cartoon T-Shirt', 'Soft cotton, fun prints', 19.99,
            'Kids', '/images/tshirt.jpg', true, 'XS'),
        new Product(4L, 'Slim Fit Chinos', 'Modern cut, beige', 49.99,
            'Men', '/images/chinos.jpg', false, 'L'),
        new Product(5L, 'Wool Blend Coat', 'Elegant winter coat', 129.99,
            'Women', '/images/coat.jpg', true, 'M'),
        new Product(6L, 'Striped Polo Shirt', 'Casual and smart', 29.99,
            'Men', '/images/polo.jpg', true, 'L')
    );

    // Return ALL products
    public List<Product> getAllProducts() {
        return products;
    }

    // Find product by its ID
    public Optional<Product> getProductById(Long id) {
        return products.stream()
            .filter(p -> p.getId().equals(id))
            .findFirst();
    }
}
```



```
// Filter by category ('Men', 'Women', 'Kids')
public List<Product> getProductsByCategory(String category) {
    return products.stream()
        .filter(p -> p.getCategory().equalsIgnoreCase(category))
        .toList();
}
}
```

Why a Service Layer?

You might wonder why not call data directly from the Controller.

The Service layer exists to keep concerns separated:

- Controller handles HTTP (request/response)
- Service handles logic (filtering, calculations, validation)
- This makes code testable, reusable, and easy to maintain.

In Part 2, the Service will call a Repository which queries MySQL.

1.7 Step 3 — Creating Controllers

Controllers are the entry points for HTTP requests. Each method in a Controller maps to a URL path. The `@GetMapping` annotation says 'when the browser visits this URL, run this method'.

HomeController.java

File Location

Create file at: `src/main/java/com/stylehub/controller/HomeController.java`

```
Java - HomeController.java
package com.stylehub.controller;

import com.stylehub.service.ProductService;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

@Controller // Marks this as a Spring MVC controller
public class HomeController {

    // Spring automatically injects ProductService (Dependency Injection)
    private final ProductService productService;

    public HomeController(ProductService productService) {
        this.productService = productService;
    }

    // Maps HTTP GET / (the home page)
    @GetMapping('/')
    public String home(Model model) {
        // Model is like a 'bag' of data passed to the HTML template
        model.addAttribute('featuredProducts',
            productService.getAllProducts().subList(0, 3));
        model.addAttribute('pageTitle', 'StyleHub - Fashion for Everyone');
        return 'home'; // → resolves to templates/home.html
    }
}
```

ProductController.java

File Location

Create file at: `src/main/java/com/stylehub/controller/ProductController.java`

```
Java - ProductController.java
package com.stylehub.controller;

import com.stylehub.service.ProductService;
import org.springframework.stereotype.Controller;
```

```

import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

@Controller
@RequestMapping('/products') // Base path for all methods here
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    // GET /products → show all products
    @GetMapping
    public String listProducts(Model model) {
        model.addAttribute('products', productService.getAllProducts());
        model.addAttribute('categories', java.util.List.of('Men', 'Women', 'Kids'));
        return 'products';
    }

    // GET /products?category=Men → filter by category
    @GetMapping(params = 'category')
    public String filterByCategory(@RequestParam String category, Model model) {
        model.addAttribute('products',
productService.getProductsByCategory(category));
        model.addAttribute('categories', java.util.List.of('Men', 'Women', 'Kids'));
        model.addAttribute('selectedCategory', category);
        return 'products';
    }

    // GET /products/3 → show one product
    @GetMapping("/{id}")
    public String productDetail(@PathVariable Long id, Model model) {
        productService.getProductById(id)
            .ifPresentOrElse(
                product -> model.addAttribute('product', product),
                () -> model.addAttribute('error', 'Product not found'));
        return 'product-detail';
    }
}

```



Key Annotations Explained

@Controller	— Marks class as Spring MVC controller (returns HTML views)
@RequestMapping	— Sets a base URL prefix for all methods in the class
@GetMapping	— Maps HTTP GET request to a method
@PathVariable	— Extracts {id} from the URL path: /products/3 → id=3
@RequestParam	— Extracts query param: /products?category=Men → category='Men'
Model	— Spring object to pass data to the Thymeleaf template

1.8 Step 4 — Creating the HTML Views (Thymeleaf)

Thymeleaf allows us to write regular HTML files with special th:* attributes that Spring Boot fills with data at runtime. The HTML is rendered server-side and sent to the browser.

Base Layout — Understanding the Template

File Location

Create file at: src/main/resources/templates/home.html

```
HTML - home.html (Thymeleaf)
<!DOCTYPE html>
<html lang='en' xmlns:th='http://www.thymeleaf.org'>
<head>
  <meta charset='UTF-8'>
  <meta name='viewport' content='width=device-width, initial-scale=1.0'>
  <!-- th:text replaces the tag content with Model data -->
  <title th:text='${pageTitle}'>StyleHub</title>
  <link rel='stylesheet' th:href="@{/css/style.css}">
</head>
<body>

<!-- ===== NAVIGATION BAR ===== -->
<nav class='navbar'>
  <div class='nav-brand'>
    <a th:href="@{/}" class='logo'> StyleHub</a>
  </div>
  <ul class='nav-links'>
    <li><a th:href="@{/}">Home</a></li>
    <li><a th:href="@{/products}">Shop</a></li>
    <li><a th:href="@{/products?category=Men}">Men</a></li>
    <li><a th:href="@{/products?category=Women}">Women</a></li>
    <li><a th:href="@{/products?category=Kids}">Kids</a></li>
  </ul>
</nav>

<!-- ===== HERO SECTION ===== -->
<section class='hero'>
  <div class='hero-content'>
    <h1>New Season Arrivals</h1>
    <p>Discover the latest trends in fashion</p>
    <a th:href="@{/products}" class='btn btn-primary'>Shop Now</a>
  </div>
</section>

<!-- ===== FEATURED PRODUCTS (from Controller Model) ===== -->
<section class='featured-section'>
  <h2>Featured Products</h2>
  <div class='product-grid'>
    <!-- th:each loops over the list - like a for-each in Java -->
    <div class='product-card' th:each='product : ${featuredProducts}'>
      <img th:src='${product.imageUrl}' th:alt='${product.name}'>
```

```

        src='/images/placeholder.jpg'>
    <div class='card-body'>
        <!-- th:text inserts Java variable value into HTML -->
        <h3 th:text='${product.name}'>Product Name</h3>
        <p class='category' th:text='${product.category}'>Category</p>
        <div class='price-row'>
            <!-- th:text with formatting: ${#numbers.formatDecimal(...)} -
->
            <span class='price'
                th:text="'$' +
                ${#numbers.formatDecimal(product.price,1,2)}">
                $0.00
            </span>
            <!-- th:if shows element only when condition is true -->
            <span class='badge badge-success'
                th:if='${product.inStock}'>In Stock</span>
            <span class='badge badge-danger'
                th:unless='${product.inStock}'>Sold Out</span>
        </div>
        <!-- th:href builds dynamic URLs with product ID -->
        <a th:href="@{/products/{id}(id=${product.id})}"
            class='btn btn-secondary'>View Details</a>
    </div>
</div>
</div>
</section>

</body>
</html>

```

Thymeleaf Quick Reference

Thymeleaf Attribute	What It Does / Example
th:text='\${var}'	Sets tag text content from Model variable
th:each='item : \${list}'	Loops over a list — renders the element once per item
th:href="@{/path}"	Creates a safe, context-relative URL link
th:href="@{/p/{id}(id=\${p.id})}"	Creates a URL with path variable: /p/3
th:if='\${condition}'	Shows element only if condition is true
th:unless='\${condition}'	Shows element only if condition is false (opposite of th:if)
th:src='\${url}'	Sets image src from variable
th:class='\${cssClass}'	Dynamically sets CSS class from variable
\${#numbers.formatDecimal(v,1,2)}	Formats number: 59.9 → '59.90'
\${#strings.toUpperCase(str)}	String utility: 'hello' → 'HELLO'

1.9 Step 5 — Products Page (products.html)

File Location

Create file at: src/main/resources/templates/products.html

```
HTML - products.html
<!DOCTYPE html>
<html lang='en' xmlns:th='http://www.thymeleaf.org'>
<head>
  <meta charset='UTF-8'>
  <title>Products - StyleHub</title>
  <link rel='stylesheet' th:href="@{/css/style.css}">
</head>
<body>

<!-- Include navigation (copy the nav from home.html) -->

<main class='container'>
  <div class='page-header'>
    <h1>Our Collection</h1>
    <p th:if='${selectedCategory}'
      th:text="'Showing: ' + ${selectedCategory}">Category</p>
  </div>

  <!-- CATEGORY FILTER BUTTONS -->
  <div class='filter-bar'>
    <a th:href="@{/products}" class='filter-btn'
      th:classappend="${selectedCategory == null} ? ' active' : ''">All</a>
    <!-- th:each can also iterate over a simple list of strings -->
    <a th:each='cat : ${categories}'
      th:href="@{/products(category=${cat})}"
      th:text='${cat}'
      class='filter-btn'
      th:classappend="${cat == selectedCategory} ? ' active' : ''">
      Category
    </a>
  </div>

  <!-- PRODUCT GRID -->
  <div class='product-grid'>
    <div class='product-card' th:each='product : ${products}'>
      <div class='card-image'>
        <img th:src='${product.imageUrl}' th:alt='${product.name}'
          src='/images/placeholder.jpg'>
        <span class='size-badge' th:text='${product.size}'>M</span>
      </div>
      <div class='card-body'>
        <span class='category-tag'
          th:text='${product.category}'>Category</span>
        <h3 th:text='${product.name}'>Product Name</h3>
        <p class='description' th:text='${product.description}'>Desc</p>
        <div class='price-row'>
```

```

        <span class='price'
            th:text="'$' +
${#numbers.formatDecimal(product.price,1,2)}">
            $0.00
        </span>
        <span th:if='${product.inStock}' class='badge badge-success'>
            In Stock
        </span>
        <span th:unless='${product.inStock}' class='badge badge-
danger'>
            Sold Out
        </span>
    </div>
    <a th:href="@{/products/{id}(id=${product.id})}"
        class='btn btn-primary'>View Details</a>
</div>
</div>
</div>

<!-- EMPTY STATE: shown if no products match filter -->
<div class='empty-state' th:if='${#lists.isEmpty(products)}'>
    <p>No products found in this category.</p>
    <a th:href="@{/products}" class='btn btn-secondary'>Show All</a>
</div>

</main>
</body>
</html>

```

1.10 Step 6 — CSS Styling (style.css)

Create a modern, clean stylesheet for StyleHub. This goes in the static resources folder, which Spring Boot serves directly to the browser without any processing.

File Location

Create file at: `src/main/resources/static/css/style.css`

```
CSS - style.css
/* ===== CSS VARIABLES (Design Tokens) ===== */
:root {
  --primary:    #2E7D32;    /* Dark green */
  --secondary:  #4CAF50;    /* Light green */
  --accent:     #FF6F00;    /* Orange CTA */
  --text:       #212121;
  --text-muted: #757575;
  --bg:         #FAFAFA;
  --white:      #FFFFFF;
  --shadow:     0 2px 12px rgba(0,0,0,0.1);
  --radius:     8px;
}

/* ===== RESET & BASE ===== */
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: 'Segoe UI', sans-serif; background: var(--bg); color: var(--text); }
a { text-decoration: none; color: inherit; }

/* ===== NAVBAR ===== */
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 1rem 2rem;
  background: var(--white);
  box-shadow: var(--shadow);
  position: sticky;
  top: 0;
  z-index: 100;
}
.logo { font-size: 1.5rem; font-weight: 700; color: var(--primary); }
.nav-links { display: flex; list-style: none; gap: 1.5rem; }
.nav-links a { font-weight: 500; transition: color 0.2s; }
.nav-links a:hover { color: var(--secondary); }

/* ===== HERO SECTION ===== */
.hero {
  background: linear-gradient(135deg, var(--primary) 0%, var(--secondary) 100%);
  color: white;
  padding: 5rem 2rem;
  text-align: center;
}
```



```

.hero h1 { font-size: 3rem; margin-bottom: 1rem; }
.hero p { font-size: 1.2rem; margin-bottom: 2rem; opacity: 0.9; }

/* ===== BUTTONS ===== */
.btn {
  display: inline-block;
  padding: 0.75rem 1.5rem;
  border-radius: var(--radius);
  font-weight: 600;
  cursor: pointer;
  transition: all 0.2s;
  border: none;
}
.btn-primary { background: var(--accent); color: white; }
.btn-secondary { background: var(--primary); color: white; }
.btn:hover { transform: translateY(-2px); box-shadow: var(--shadow); }

/* ===== PRODUCT GRID ===== */
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 1.5rem;
  padding: 2rem;
}

/* ===== PRODUCT CARD ===== */
.product-card {
  background: var(--white);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  overflow: hidden;
  transition: transform 0.2s;
}
.product-card:hover { transform: translateY(-4px); }
.product-card img { width: 100%; height: 220px; object-fit: cover; }
.card-body { padding: 1rem; }
.card-body h3 { margin-bottom: 0.3rem; font-size: 1.1rem; }

/* ===== PRICE & BADGES ===== */
.price { font-size: 1.3rem; font-weight: 700; color: var(--primary); }
.price-row { display: flex; align-items: center; gap: 0.5rem; margin: 0.5rem 0; }
.badge { padding: 0.25rem 0.6rem; border-radius: 20px; font-size: 0.8rem; font-weight: 600; }
.badge-success { background: #E8F5E9; color: #2E7D32; }
.badge-danger { background: #FFEBEE; color: #B71C1C; }

/* ===== FILTER BAR ===== */
.filter-bar { display: flex; gap: 1rem; padding: 1rem 2rem; flex-wrap: wrap; }
.filter-btn { padding: 0.5rem 1.2rem; border-radius: 20px; border: 2px solid var(--primary);
  color: var(--primary); font-weight: 600; transition: all 0.2s; }
.filter-btn:hover,
.filter-btn.active { background: var(--primary); color: white; }

```

```
/* ===== CONTAINER & UTILITIES ===== */
.container { max-width: 1200px; margin: 0 auto; padding: 1rem; }
.page-header { padding: 2rem; }
.page-header h1 { font-size: 2rem; color: var(--primary); }
.featured-section { padding: 2rem; }
.featured-section h2 { font-size: 1.8rem; margin-bottom: 1rem; text-align: center;
}
.empty-state { text-align: center; padding: 3rem; color: var(--text-muted); }
```

1.11 Part 1 — Summary & Testing

Before moving to Part 2, let's verify everything works correctly. Start the application and test each URL:

URL to Test	Expected Result
http://localhost:8080/	Home page with hero section and 3 featured products
http://localhost:8080/products	Grid of all 6 products
http://localhost:8080/products?category=Men	Only Men's products (3 items)
http://localhost:8080/products?category=Women	Only Women's products (2 items)
http://localhost:8080/products/1	Detail page for 'Classic Denim Jacket'
http://localhost:8080/products/99	Error message: Product not found

Part 1 — What You Learned

- ✓ Spring Boot project structure and setup
- ✓ MVC pattern: Model → Service → Controller → View
- ✓ @Controller, @GetMapping, @PathVariable, @RequestParam annotations
- ✓ Model object to pass data to templates
- ✓ Thymeleaf: th:each, th:text, th:if, th:href, th:classappend
- ✓ CSS: CSS Variables, Flexbox, CSS Grid, responsive card layout
- ✓ Lombok: @Data, @AllArgsConstructor to reduce boilerplate
- ✓ Static in-memory data (Java List) as a database placeholder

PART 2

Database Integration — Spring Data JPA & MySQL

2.1 What is Spring Data JPA?

JPA (Java Persistence API) is a specification for how Java objects map to database tables. Spring Data JPA is Spring's implementation that adds powerful repository abstractions. Instead of writing SQL manually, you describe what you want and Spring generates the queries.

Technology	Role
JPA	Specification: how Java objects map to DB tables (@Entity, @Column, @Id)
Hibernate	JPA implementation — the engine that generates actual SQL behind the scenes
Spring Data JPA	Repository layer — findAll(), findById(), save(), delete() auto-generated
MySQL	The relational database where data is actually stored
JDBC	Low-level connection between Java and MySQL (managed by Spring)

New Dependencies to Add to pom.xml

Open pom.xml and add these dependencies inside the <dependencies> section:

```
XML — pom.xml additions
<!-- MySQL JDBC Driver -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>

<!-- Spring Data JPA -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Spring Boot Validation (for form validation) -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
```

```
</dependency>
```

2.2 Database Setup (MySQL)

Start MySQL and create the database and user for StyleHub. Open MySQL Workbench or run the following SQL commands in your terminal:

```
SQL - MySQL Setup

-- Connect as root first:
mysql -u root -p

-- Create a dedicated database for our app:
CREATE DATABASE stylehub_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- Create a dedicated user (better practice than using root):
CREATE USER 'stylehub_user'@'localhost' IDENTIFIED BY 'StyleHub@2024';

-- Grant permissions:
GRANT ALL PRIVILEGES ON stylehub_db.* TO 'stylehub_user'@'localhost';
FLUSH PRIVILEGES;

-- Verify:
SHOW DATABASES;

-- You should see: stylehub_db in the list
```

Configure application.properties

File Location

Edit: src/main/resources/application.properties

```
Properties - application.properties

# ===== DATABASE CONNECTION =====
spring.datasource.url=jdbc:mysql://localhost:3306/stylehub_db
spring.datasource.username=stylehub_user
spring.datasource.password=StyleHub@2024
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# ===== JPA / HIBERNATE =====
# create-drop: drops and recreates tables on every start (use in dev only!)
# update: keeps existing data, adds new columns if model changes
# validate: only validates schema, never changes it (use in production)
spring.jpa.hibernate.ddl-auto=update

# Show generated SQL in console (useful for learning/debugging)
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# MySQL dialect - tells Hibernate to generate MySQL-compatible SQL
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

# ===== APP CONFIG =====
```

```
spring.application.name=StyleHub  
server.port=8080
```

Security Note

Never commit database passwords to version control (Git).

Use environment variables or Spring profiles in production.

Example: `spring.datasource.password=${DB_PASSWORD}`

2.3 Step 1 — Converting Product to a JPA Entity

We need to annotate our Product class so Hibernate knows how to map it to a MySQL table. We also switch from a simple Long id to a database-generated one.

File to Modify

Edit: src/main/java/com/stylehub/model/Product.java

```
Java - Product.java (JPA Entity)
package com.stylehub.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;

// @Entity: tells JPA this class maps to a database table
@Entity
// @Table: specifies the exact table name in MySQL
@Table(name = 'products')
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Product {

    // @Id: marks this as the primary key
    // @GeneratedValue: auto-increment in MySQL (1, 2, 3, ...)
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    // @Column: maps field to a DB column, with constraints
    @Column(nullable = false, length = 150)
    @NotBlank(message = 'Product name is required')
    private String name;

    @Column(columnDefinition = 'TEXT')
    private String description;

    @Column(nullable = false)
    @Positive(message = 'Price must be positive')
    private double price;

    @Column(nullable = false, length = 50)
    @NotBlank(message = 'Category is required')
    private String category;

    @Column(name = 'image_url')
    private String imageUrl;

    @Column(name = 'in_stock')
```



```

private boolean inStock = true;

@Column(length = 10)
private String size;

// ===== New field - stock quantity =====
@Column(name = 'stock_quantity')
@Min(value = 0, message = 'Stock cannot be negative')
private int stockQuantity = 0;
}

```

Annotation	What It Does
@Entity	Marks class as JPA entity → creates DB table
@Table(name='products')	Names the MySQL table 'products'
@Id	Marks field as primary key
@GeneratedValue(IDENTITY)	Auto-increment: MySQL assigns 1, 2, 3...
@Column(nullable=false)	SQL: NOT NULL constraint
@Column(length=150)	SQL: VARCHAR(150)
@Column(columnDefinition='TEXT')	SQL: TEXT type (unlimited length)
@NotBlank	Validation: field cannot be empty (Spring Validation)
@Positive	Validation: number must be > 0

2.4 Step 2 — Creating the Repository

The Repository is the data access layer. By extending `JpaRepository`, we get dozens of database operations for free — no SQL required.

Create File

Create: `src/main/java/com/stylehub/repository/ProductRepository.java`

```
Java - ProductRepository.java
package com.stylehub.repository;

import com.stylehub.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

// JpaRepository<EntityType, PrimaryKeyType>
// Spring AUTO-GENERATES the implementation of this interface!
@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Spring derives SQL from the method name!
    // SELECT * FROM products WHERE category = ?
    List<Product> findByCategory(String category);

    // SELECT * FROM products WHERE in_stock = true
    List<Product> findByInStockTrue();

    // SELECT * FROM products WHERE category = ? AND in_stock = true
    List<Product> findByCategoryAndInStockTrue(String category);

    // Search by name containing keyword (case-insensitive)
    // SELECT * FROM products WHERE LOWER(name) LIKE %keyword%
    List<Product> findByNameContainingIgnoreCase(String keyword);

    // Custom JPQL query (Java Persistence Query Language)
    @Query('SELECT p FROM Product p WHERE p.price BETWEEN :min AND :max ORDER BY p.price')
    List<Product> findByPriceBetween(double min, double max);

    // Count by category
    long countByCategory(String category);
}
```

🔮 Spring Data Magic: Derived Query Methods

Spring reads your method name and generates SQL automatically:

`findByCategory(cat)` → `WHERE category = cat`

findByInStockTrue() → WHERE in_stock = 1

findByPriceLessThan(price) → WHERE price < price

findByNameContainingIgnoreCase → WHERE LOWER(name) LIKE %val%

findByCategoryOrderByPriceDesc → WHERE category=? ORDER BY price DESC

No SQL needed — just name your method correctly!

Built-in JpaRepository Methods (Free!)

Method	SQL Equivalent
findAll()	SELECT * FROM products
findById(1L)	SELECT * FROM products WHERE id = 1
save(product)	INSERT or UPDATE (detects automatically)
deleteById(1L)	DELETE FROM products WHERE id = 1
count()	SELECT COUNT(*) FROM products
existsById(1L)	SELECT COUNT(*) > 0 WHERE id = 1
findAll(Sort.by('price'))	SELECT * FROM products ORDER BY price
findAll(PageRequest.of(0, 10))	SELECT * FROM products LIMIT 10 OFFSET 0

2.5 Step 3 — Updating the Service Layer

Now we update ProductService to use the Repository (database) instead of the static list. This is the only change needed — the Controller and Views stay the same!

File to Modify

Edit: src/main/java/com/stylehub/service/ProductService.java

```
Java - ProductService.java (Updated)
package com.stylehub.service;

import com.stylehub.model.Product;
import com.stylehub.repository.ProductRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;

@Service
public class ProductService {

    // Now we inject the Repository instead of using a static list
    private final ProductRepository productRepository;

    public ProductService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // ===== READ OPERATIONS =====

    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }

    public Optional<Product> getProductById(Long id) {
        return productRepository.findById(id);
    }

    public List<Product> getProductsByCategory(String category) {
        return productRepository.findByCategory(category);
    }

    public List<Product> searchProducts(String keyword) {
        return productRepository.findByNameContainingIgnoreCase(keyword);
    }

    // ===== WRITE OPERATIONS =====

    // @Transactional: ensures the operation is atomic (all or nothing)
    @Transactional
```

```
public Product saveProduct(Product product) {
    // save() handles both INSERT (new) and UPDATE (existing)
    return productRepository.save(product);
}

@Transactional
public void deleteProduct(Long id) {
    productRepository.deleteById(id);
}

@Transactional
public Product updateProduct(Long id, Product updatedProduct) {
    return productRepository.findById(id)
        .map(existing -> {
            existing.setName(updatedProduct.getName());
            existing.setPrice(updatedProduct.getPrice());
            existing.setCategory(updatedProduct.getCategory());
            existing.setDescription(updatedProduct.getDescription());
            existing.setInStock(updatedProduct.isInStock());
            existing.setStockQuantity(updatedProduct.getStockQuantity());
            return productRepository.save(existing);
        })
        .orElseThrow(() -> new RuntimeException('Product not found: ' + id));
}
```

2.6 Step 4 — Admin CRUD Controller

We need an admin interface to create, edit, and delete products. We'll add a new AdminController with full CRUD operations.

Create File

Create: src/main/java/com/stylehub/controller/AdminController.java

```
Java - AdminController.java
package com.stylehub.controller;

import com.stylehub.model.Product;
import com.stylehub.service.ProductService;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.servlet.mvc.support.RedirectAttributes;

@Controller
@RequestMapping('/admin')
public class AdminController {

    private final ProductService productService;

    public AdminController(ProductService productService) {
        this.productService = productService;
    }

    // GET /admin → show all products (admin dashboard)
    @GetMapping
    public String dashboard(Model model) {
        model.addAttribute('products', productService.getAllProducts());
        model.addAttribute('totalCount', productService.getAllProducts().size());
        return 'admin/dashboard';
    }

    // GET /admin/products/new → show empty form
    @GetMapping('/products/new')
    public String showCreateForm(Model model) {
        model.addAttribute('product', new Product());
        model.addAttribute('categories', java.util.List.of('Men', 'Women', 'Kids'));
        return 'admin/product-form';
    }

    // POST /admin/products → save new product to DB
    @PostMapping('/products')
    public String createProduct(
        @Valid @ModelAttribute Product product, // Validates annotations
        BindingResult result, // Holds validation errors
    ) {
    }
}
```

```

        RedirectAttributes redirectAttrs) {
    if (result.hasErrors()) {
        return 'admin/product-form'; // Return to form with errors
    }
    productService.saveProduct(product);
    redirectAttrs.addFlashAttribute('successMsg', 'Product created!');
    return 'redirect:/admin';
}

// GET /admin/products/3/edit → show pre-filled edit form
@GetMapping('/products/{id}/edit')
public String showEditForm(@PathVariable Long id, Model model) {
    Product product = productService.getProductById(id)
        .orElseThrow(() -> new RuntimeException('Product not found'));
    model.addAttribute('product', product);
    model.addAttribute('categories', java.util.List.of('Men', 'Women', 'Kids'));
    return 'admin/product-form';
}

// POST /admin/products/3/update → save edited product
@PostMapping('/products/{id}/update')
public String updateProduct(
    @PathVariable Long id,
    @Valid @ModelAttribute Product product,
    BindingResult result,
    RedirectAttributes redirectAttrs) {
    if (result.hasErrors()) {
        return 'admin/product-form';
    }
    productService.updateProduct(id, product);
    redirectAttrs.addFlashAttribute('successMsg', 'Product updated!');
    return 'redirect:/admin';
}

// POST /admin/products/3/delete → delete product
@PostMapping('/products/{id}/delete')
public String deleteProduct(@PathVariable Long id, RedirectAttributes
redirectAttrs) {
    productService.deleteProduct(id);
    redirectAttrs.addFlashAttribute('successMsg', 'Product deleted!');
    return 'redirect:/admin';
}
}

```

💡 @Valid and BindingResult Explained

@Valid — triggers validation annotations on the Product object (@NotBlank, @Positive...)

BindingResult — Spring puts all validation errors here. Always put it right after @Valid.

If result.hasErrors() → return to form (show errors to user).

redirect:/admin — after success, redirect to avoid form re-submission on browser refresh.

RedirectAttributes — stores flash messages that survive a redirect.

2.7 Step 5 — Admin HTML Views

Admin Dashboard — dashboard.html

Create File

Create: src/main/resources/templates/admin/dashboard.html

```
HTML - admin/dashboard.html
<!DOCTYPE html>
<html lang='en' xmlns:th='http://www.thymeleaf.org'>
<head>
  <meta charset='UTF-8'>
  <title>Admin - StyleHub</title>
  <link rel='stylesheet' th:href="@{/css/style.css}">
  <link rel='stylesheet' th:href="@{/css/admin.css}">
</head>
<body>
<div class='admin-layout'>
  <aside class='sidebar'>
    <h2>🌿 Admin</h2>
    <nav>
      <a th:href="@{/admin}">📦 Products</a>
      <a th:href="@{/}">🏠 View Site</a>
    </nav>
  </aside>
  <main class='admin-main'>
    <div class='admin-header'>
      <h1>Products <span th:text="'(' + ${totalCount} + ')'"></span></h1>
      <a th:href="@{/admin/products/new}" class='btn btn-primary'>+ Add
Product</a>
    </div>

    <!-- Flash messages (success/error) -->
    <div class='alert alert-success' th:if='${successMsg}'
      th:text='${successMsg}'></div>

    <!-- Products table -->
    <table class='admin-table'>
      <thead>
        <tr>
          <th>ID</th><th>Name</th><th>Category</th>
          <th>Price</th><th>Stock</th><th>Actions</th>
        </tr>
      </thead>
      <tbody>
        <tr th:each='product : ${products}'>
          <td th:text='${product.id}'>1</td>
          <td th:text='${product.name}'>Name</td>
          <td th:text='${product.category}'>Category</td>
          <td th:text="'$' +
${#numbers.formatDecimal(product.price,1,2)}">$0</td>
          <td>

```



```

                <span th:if='${product.inStock}' class='badge badge-
success'>In Stock</span>
                <span th:unless='${product.inStock}' class='badge badge-
danger'>Out</span>
            </td>
            <td class='action-btns'>
                <a
th:href="@{/admin/products/{id}/edit(id=${product.id})}"
                class='btn btn-sm btn-secondary'>Edit</a>
                <!-- DELETE uses a form (POST) because HTML links only do
GET -->
                <form
th:action="@{/admin/products/{id}/delete(id=${product.id})}"
                method='post' style='display:inline'
                onsubmit="return confirm('Delete this product?')">
                <button type='submit' class='btn btn-sm btn-
danger'>Delete</button>
                </form>
            </td>
        </tr>
    </tbody>
</table>
</main>
</div>
</body>
</html>

```

Product Form — product-form.html (Create & Edit)

Create File

Create: src/main/resources/templates/admin/product-form.html

HTML — admin/product-form.html

```
<!DOCTYPE html>
<html lang='en' xmlns:th='http://www.thymeleaf.org'>
<head>
    <meta charset='UTF-8'>
    <title>Product Form — StyleHub</title>
    <link rel='stylesheet' th:href="@{/css/style.css}">
    <link rel='stylesheet' th:href="@{/css/admin.css}">
</head>
<body>
<div class='admin-layout'>
    <aside class='sidebar'>
        <h2>🌿 Admin</h2>
        <nav><a th:href="@{/admin}">📦 Products</a></nav>
    </aside>
    <main class='admin-main'>
        <h1 th:text="${product.id != null ? 'Edit Product' : 'New Product'}">Form</h1>

        <!--
            th:object='${product}' — binds entire form to the Product object
            th:action uses product.id to decide: create or update URL
        -->
        <form th:object='${product}'
            th:action="${product.id != null}
                ? @{/admin/products/{id}/update(id=${product.id})}
                : @{/admin/products}"
            method='post' class='product-form'>

            <div class='form-group'>
                <label for='name'>Product Name *</label>
                <!-- th:field='*{name}' binds to product.name -->
                <input type='text' id='name' th:field='*{name}'
                    class='form-control'>
                <!-- Shows validation error if @NotBlank fails -->
                <span class='error' th:if='${#fields.hasErrors("name")}'
                    th:errors='*{name}'>Error</span>
            </div>

            <div class='form-group'>
                <label for='description'>Description</label>
                <textarea id='description' th:field='*{description}'
                    class='form-control' rows='4'></textarea>
            </div>

            <div class='form-row'>
                <div class='form-group'>
                    <label for='price'>Price ($) *</label>
```

```

        <input type='number' id='price' th:field='*{price}'
            step='0.01' min='0' class='form-control'>
        <span class='error' th:if='${#fields.hasErrors("price")}'
            th:errors='*{price}'>Error</span>
    </div>
    <div class='form-group'>
        <label for='category'>Category *</label>
        <select id='category' th:field='*{category}' class='form-
control'>
            <option value=''>-- Select --</option>
            <option th:each='cat : ${categories}'
                th:value='${cat}'
th:text='${cat}'>Category</option>
        </select>
    </div>
</div>

<div class='form-row'>
    <div class='form-group'>
        <label for='size'>Size</label>
        <select id='size' th:field='*{size}' class='form-control'>
            <option th:each='s : ${ { "XS", "S", "M", "L", "XL", "XXL" } }'
                th:value='${s}' th:text='${s}'>S</option>
        </select>
    </div>
    <div class='form-group'>
        <label for='stockQuantity'>Stock Quantity</label>
        <input type='number' id='stockQuantity'
th:field='*{stockQuantity}'
            min='0' class='form-control'>
    </div>
</div>

<div class='form-group checkbox-group'>
    <input type='checkbox' id='inStock' th:field='*{inStock}'>
    <label for='inStock'>In Stock</label>
</div>

<div class='form-group'>
    <label for='imageUrl'>Image URL</label>
    <input type='text' id='imageUrl' th:field='*{imageUrl}'
        class='form-control' placeholder='/images/product.jpg'>
</div>

<div class='form-actions'>
    <button type='submit' class='btn btn-primary'>
        <span th:text='${product.id != null ? 'Update Product' :
'Create Product'}">Save</span>
    </button>
    <a th:href="@{/admin}" class='btn btn-secondary'>Cancel</a>
</div>
</form>
</main>
</div>
</body>

```

```
</html>
```

2.8 Step 6 — Database Seeding (Initial Data)

We need sample data in MySQL when the app starts. Spring Boot supports a `data.sql` file for initial data, or we can use a `DataInitializer` component.

Create File

Create: `src/main/java/com/stylehub/config/DataInitializer.java`

```
Java - DataInitializer.java
package com.stylehub.config;

import com.stylehub.model.Product;
import com.stylehub.repository.ProductRepository;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

// @Component makes this a Spring bean - runs on startup
// CommandLineRunner.run() is called after app starts
@Component
public class DataInitializer implements CommandLineRunner {

    private final ProductRepository repo;

    public DataInitializer(ProductRepository repo) {
        this.repo = repo;
    }

    @Override
    public void run(String... args) {
        // Only seed if table is empty (avoid duplicates on restart)
        if (repo.count() == 0) {
            repo.save(new Product(null, 'Classic Denim Jacket',
                'Timeless blue denim, perfect for any season',
                59.99, 'Men', '/images/jacket.jpg', true, 'M', 25));
            repo.save(new Product(null, 'Floral Summer Dress',
                'Light chiffon fabric with vibrant floral print',
                44.99, 'Women', '/images/dress.jpg', true, 'S', 18));
            repo.save(new Product(null, 'Kids Cartoon T-Shirt',
                'Soft 100% cotton with fun cartoon prints',
                19.99, 'Kids', '/images/tshirt.jpg', true, 'XS', 40));
            repo.save(new Product(null, 'Slim Fit Chinos',
                'Modern cut chinos in elegant beige',
                49.99, 'Men', '/images/chinos.jpg', false, 'L', 0));
            repo.save(new Product(null, 'Wool Blend Coat',
                'Elegant winter coat, warm and stylish',
                129.99, 'Women', '/images/coat.jpg', true, 'M', 12));
            repo.save(new Product(null, 'Striped Polo Shirt',
                'Casual yet smart striped polo',
                29.99, 'Men', '/images/polo.jpg', true, 'L', 30));
            System.out.println('☑ Database seeded with 6 products');
        }
    }
}
```

}

}

2.9 Entity Relationships — Category Entity

A real e-commerce app needs proper relationships between tables. Let's add a Category entity and link it to Product using a Many-to-One relationship (many products belong to one category).

Create File

Create: src/main/java/com/stylehub/model/Category.java

```
Java - Category.java
package com.stylehub.model;

import jakarta.persistence.*;
import lombok.*;
import java.util.List;

@Entity
@Table(name = 'categories')
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 80)
    private String name;    // 'Men', 'Women', 'Kids'

    private String description;
    private String iconEmoji; // '👕', '👗', '👶'

    // One Category has MANY Products
    // mappedBy = 'category': the 'category' field in Product owns the FK
    // cascade = ALL: if category deleted, products are deleted too (be careful!)
    // fetch = LAZY: only load products when explicitly accessed (performance)
    @OneToMany(mappedBy = 'category', fetch = FetchType.LAZY)
    private List<Product> products;
}
```

Update Product.java to reference Category entity:

```
Java - Product.java (relationship update)
// Replace the String category field in Product.java with:

// @ManyToOne: many products → one category
// @JoinColumn: the FK column in 'products' table is 'category_id'
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = 'category_id')
private Category category;
```

```
// Hibernate generates this SQL:  
// ALTER TABLE products ADD COLUMN category_id BIGINT;  
// ALTER TABLE products ADD FOREIGN KEY (category_id) REFERENCES categories(id);
```

JPA Relationship	Meaning / Use Case
@ManyToOne	Many products → one category (most common)
@OneToMany	One category → many products
@OneToOne	One product → one product detail page
@ManyToMany	Products ↔ Tags (one product has many tags, one tag on many products)
FetchType.LAZY	Load related data only when accessed (better performance)
FetchType.EAGER	Always load related data immediately (use carefully)
CascadeType.ALL	Operations on parent cascade to children (save, delete, etc.)

2.10 Full URL Mapping Reference

Here is a complete summary of all routes in the StyleHub application:

Method + URL	Description
GET /	Home page with featured products
GET /products	All products grid
GET /products?category=Men	Filter products by category
GET /products/{id}	Single product detail page
GET /admin	Admin dashboard — list all products
GET /admin/products/new	Show empty form to create product
POST /admin/products	Save new product to MySQL
GET /admin/products/{id}/edit	Show pre-filled edit form
POST /admin/products/{id}/update	Save updated product to MySQL
POST /admin/products/{id}/delete	Delete product from MySQL

2.11 Part 2 — Testing Checklist

1. Start MySQL service and verify it is running on port 3306
2. Run: `./mvnw spring-boot:run` — check console for Hibernate SQL output
3. Verify: Tomcat started on port 8080
4. Open: `http://localhost:8080` — home page should load (data from DB)
5. Open: `http://localhost:8080/admin` — admin dashboard should show 6 products
6. Test CREATE: click '+ Add Product' → fill form → submit → verify it appears
7. Test EDIT: click Edit → modify price → submit → verify change in table
8. Test DELETE: click Delete → confirm → verify product removed
9. Verify MySQL: run `'SELECT * FROM products;'` in MySQL Workbench — should match UI
10. Test validation: submit form with empty Name field → error message should appear

Part 2 — What You Learned

- ✓ Spring Data JPA: `@Entity`, `@Id`, `@GeneratedValue`, `@Column` annotations
- ✓ Repository pattern: `JpaRepository` gives you CRUD for free
- ✓ Derived query methods: `findByCategory`, `findByNameContainingIgnoreCase`
- ✓ Custom `@Query` with JPQL

✓ @Transactional for write operations

✓ Full CRUD with AdminController: @PostMapping, @ModelAttribute, @Valid

✓ Thymeleaf form binding: th:object, th:field, th:errors

✓ Entity relationships: @ManyToOne, @OneToMany, @JoinColumn

✓ Database seeding with CommandLineRunner

✓ Flash messages with RedirectAttributes

✓ application.properties: datasource, JPA, Hibernate configuration

2.12 Troubleshooting Guide

Error	Solution
Access denied for user 'stylehub_user'	Check username/password in application.properties match MySQL
Communications link failure Table 'products' doesn't exist	MySQL is not running — start MySQL service Set ddl-auto=update and restart — Hibernate creates it
Whitelabel Error Page (404)	URL doesn't match any @GetMapping — check controller paths
Thymeleaf template not found	Template name returned by controller must match file in templates/
NullPointerException on product	getProductById returns Optional — use .orElseThrow()
Port 8080 already in use	Add server.port=8081 to application.properties
Lombok not working	Install Lombok plugin in IntelliJ + enable annotation processing

2.13 Next Steps & Further Learning

 What to Explore Next
Spring Security — Add login/logout for admin panel (@EnableWebSecurity)
Shopping Cart — Implement cart with Session or dedicated Cart entity
Order Management — Order entity with @ManyToMany for products
REST API — Add @RestController for JSON API (mobile apps, SPAs)
Pagination — PageRequest for large product lists (Pageable)
Image Upload — MultipartFile for uploading product images
Deployment — Package with 'mvn package' → deploy to AWS/Heroku/Railway
Testing — @SpringBootTest, @WebMvcTest, @DataJpaTest for unit/integration tests

— End of StyleHub Spring Boot Tutorial —